

MOUNT KENYA UNIVERSITY

School of Computing and Informatics

BIT4107: Mobile Application Development

Final Capstone Project Report

RETAIL ANALYTICS ENGINE

A Mobile Inventory, Sales, and Business Intelligence System for General Retail Businesses

Prepared by	Noelle Blessing Ochieng
Registration Number	BIT/2024/73309
Programme	Bachelor of Information Technology
Course	BIT4107 – Mobile Application Development
Lecturer	Mr. Michael Nyoro
Academic Year	Year 3, Semester 2, 2026
Repository	lionoble20-ctrl/-BIT4107-Mobile-Development
App Version	v1.1.0 (Android, built with Flutter)

Contents

Abstract	3
1. Introduction	3
2. Problem Statement	3
3. Objectives	4
3.1 General Objective	4
3.2 Specific Objectives	4
4. Literature Review	4
5. System Analysis.....	4
5.1 Functional Requirements	4
5.2 Non-Functional Requirements.....	5
5.3 Target Users	5
6. System Design	5
6.1 Use Case Diagram.....	5
6.2 Class Diagram	6
6.3 Entity-Relationship Diagram	7
7. Implementation	9
7.1 Authentication and Roles	9
7.2 Inventory and Sales	10
7.3 Profit and Loss Dashboard	11
7.4 Payments.....	11
7.5 AI Business Assistant.....	12
7.6 Currency and Market Context	12
7.7 Device Features	13
8. Testing	15
9. Performance Evaluation	15
10. Challenges Encountered.....	16
11. Recommendations.....	17
12. Conclusion	17
13. References	17
14. Appendices	17

Abstract

The Retail Analytics Engine is a cross-platform mobile application built with Flutter to help small and medium retail businesses manage inventory, record sales, and understand their own numbers without needing a dedicated accountant or a desktop point-of-sale system. Unlike single-purpose stock apps, it is designed as a general-purpose retail tool: any category of retail business — from groceries to hardware to clothing to electronics — can configure it around their own products, currency, and business type during setup. The system provides secure role-based authentication (Merchant and Operator accounts), full inventory CRUD backed by a local SQLite database (with a Sembast fallback for the web build), integrated M-Pesa payment recording through IntaSend, live currency conversion and macroeconomic context through the Open Exchange Rates and World Bank APIs, an AI-powered business assistant built on Groq's Llama 3.3 model, and device-level features including camera-based product capture and GPS location tagging. Across the semester the project moved through structured phases of design, implementation, testing, debugging, and performance optimization, culminating in a signed, deployable Android release. This report documents the system's design, implementation, evaluation, and the lessons drawn from building it as a continuous, iterative project rather than a one-off assignment.

1. Introduction

Many small retail businesses — regardless of what they sell — still track stock and sales with paper ledgers, memory, or, at best, a spreadsheet that nobody updates consistently. This creates blind spots: operators don't always know what's actually in stock, what is and isn't profitable, or how their prices compare once currency and inflation are accounted for. The Retail Analytics Engine was built to close that gap on a device retail operators already carry — their phone — without requiring specialised hardware or a monthly software subscription.

The system was developed as a continuous project across a full semester of BIT4107, with each week's coursework adding a working layer onto the same codebase: authentication and data modelling first, then the core inventory and sales workflows, then payments, AI features, and device integrations, and finally testing, performance optimization, and deployment. That structure is reflected in this report — the Implementation, Testing, and Performance sections describe real, working features rather than a theoretical design that was never built.

2. Problem Statement

Retail operators across many different trades share a common set of problems that generic spreadsheet or paper-based tracking does not solve well:

- Stock levels are not known in real time, leading to overselling, stockouts, or dead stock sitting unnoticed.
- Profit margins are rarely calculated accurately per product, so operators cannot tell which items are actually worth stocking.
- Sales and payments are recorded separately from inventory, if at all, making it hard to reconcile what was sold against what remains.
- Local currency figures are not contextualised against inflation or exchange rate movement, which matters for businesses that import stock or price against a foreign currency.
- Existing point-of-sale software is often built for large retailers, priced accordingly, and not designed to run offline-first on a low-to-mid-range Android phone.

The Retail Analytics Engine addresses these problems directly, and — critically — is built so that no part of its data model, workflow, or UI assumes a specific category of retail. A business type field,

configurable currency, and free-text product categories mean the same app serves a grocery kiosk, a hardware store, a boutique, or an electronics shop equally well.

3. Objectives

3.1 General Objective

To design, develop, test, and deploy a mobile application that gives general retail businesses real-time visibility into their inventory, sales, and profitability, regardless of what they sell.

3.2 Specific Objectives

- To implement secure, role-based user authentication with self-service and admin-assisted password recovery.
- To build a relational data model and CRUD workflows for products, sales, and business settings.
- To integrate a mobile payment provider (M-Pesa via IntaSend) for recording real transactions.
- To integrate external APIs for live currency conversion and macroeconomic context (Open Exchange Rates, World Bank).
- To provide an AI-driven business assistant capable of answering stock and sales questions in natural language.
- To incorporate at least two native device features (camera, GPS) into real workflows rather than as demos.
- To validate correctness through structured testing and debugging, and to profile and optimize runtime performance.
- To package and sign the application for deployment as an installable Android release.

4. Literature Review

Commercial point-of-sale and inventory tools (e.g. Square, Loyverse, Zoho Inventory) offer comprehensive feature sets but are generally built around continuous internet connectivity, subscription pricing, and hardware add-ons (barcode scanners, receipt printers) that are not always practical for small operators in markets where connectivity and hardware budgets are constrained. Lightweight offline-first alternatives exist but typically stop at basic stock counts and do not extend into payment recording, AI-assisted decision support, or localized economic context. The Retail Analytics Engine sits between these two categories: it keeps the offline-first, low-hardware-dependency approach of lightweight stock apps, while adding the payment integration, AI assistant, and analytics depth normally reserved for paid commercial platforms.

5. System Analysis

5.1 Functional Requirements

- Users shall be able to register, log in, and recover access via a security question or admin-assisted reset.
- Merchants shall be able to manage Operator accounts; both roles shall manage inventory and record sales.
- The system shall support full CRUD on products, and shall record every sale against the product it affects.
- The system shall calculate and display profit margins, stock alerts, and a real-time profit and loss dashboard.

- The system shall support recording payments through M-Pesa (via IntaSend) in both simulated and live modes.
- The system shall display live currency exchange rates and relevant macroeconomic indicators.
- The system shall provide an AI chat interface that can answer questions using the user's own live inventory data.
- The system shall allow product photo capture and GPS location tagging as part of stock records.

5.2 Non-Functional Requirements

- Performance: the UI shall sustain close to 60 FPS during normal use, with no unbounded memory growth during extended navigation.
- Security: passwords shall never be stored in plaintext; SHA-256 hashing is applied before any credential is persisted.
- Availability: core inventory and sales functions shall work fully offline, since the primary data store is local (SQLite/Sembast).
- Usability: the interface shall support both dark and light themes and follow Material Design conventions.

5.3 Target Users

Two roles are modelled: a Merchant, who owns the business account and can manage Operator accounts in addition to day-to-day functions; and an Operator, a staff account with access to the operational features (inventory, sales, dashboard, advisory, assistant) but not account administration.

6. System Design

6.1 Use Case Diagram

The diagram below summarises the actions available to each role. Both roles share the core operational use cases; account administration (managing Operator accounts) is restricted to the Merchant role.

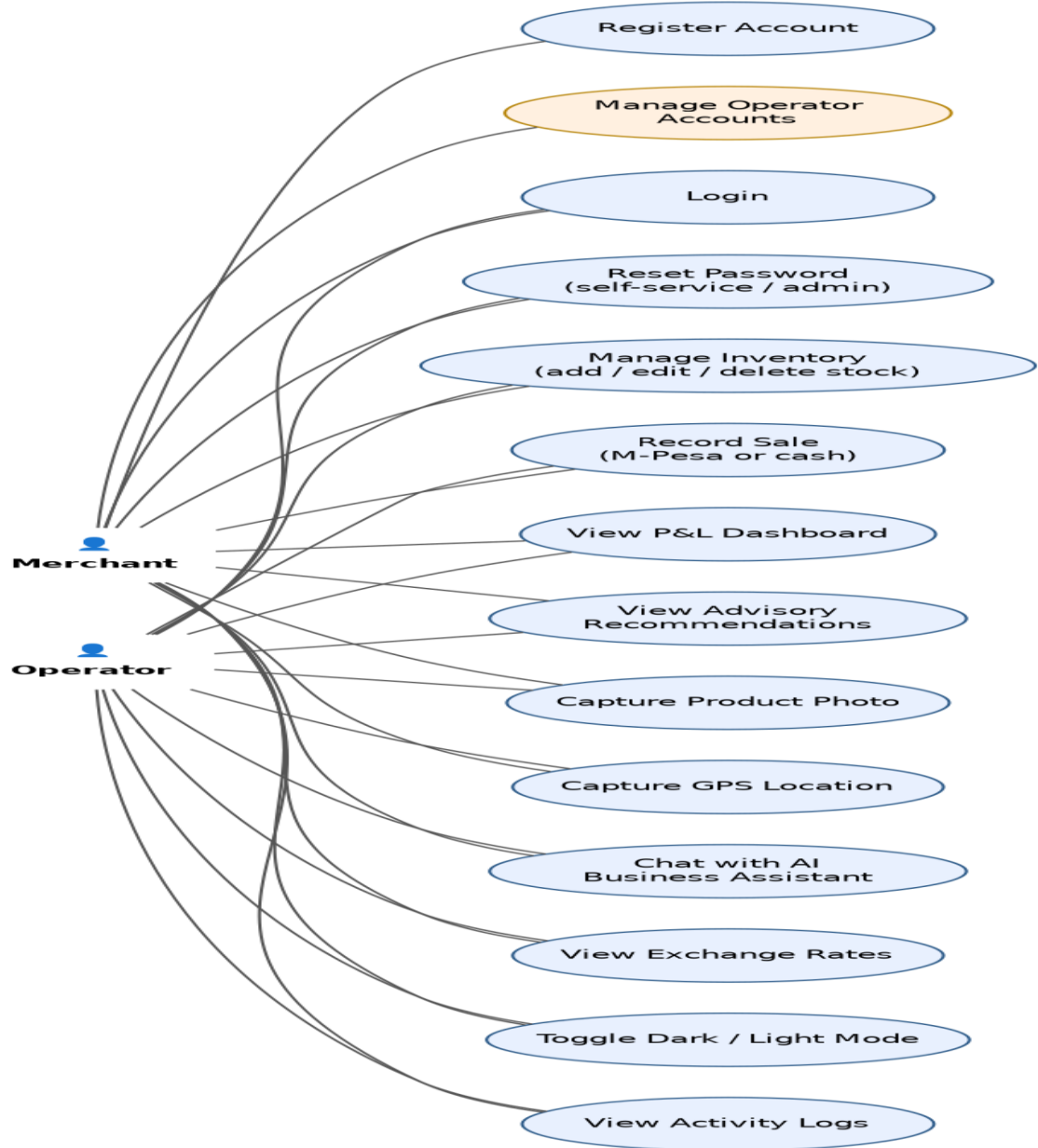


Figure 1 – Use Case Diagram: Merchant and Operator interactions with the system.

6.2 Class Diagram

The diagram below shows the core data classes (InventoryItem, SaleRecord), the persistence layer (DatabaseHelper), and the supporting service classes introduced during the OOP-focused stage of the project (ValidatorService, EventLoggerService, InputHandlerService, GestureService), alongside the external-facing services (GroqService for the AI assistant, PaymentService for M-Pesa).

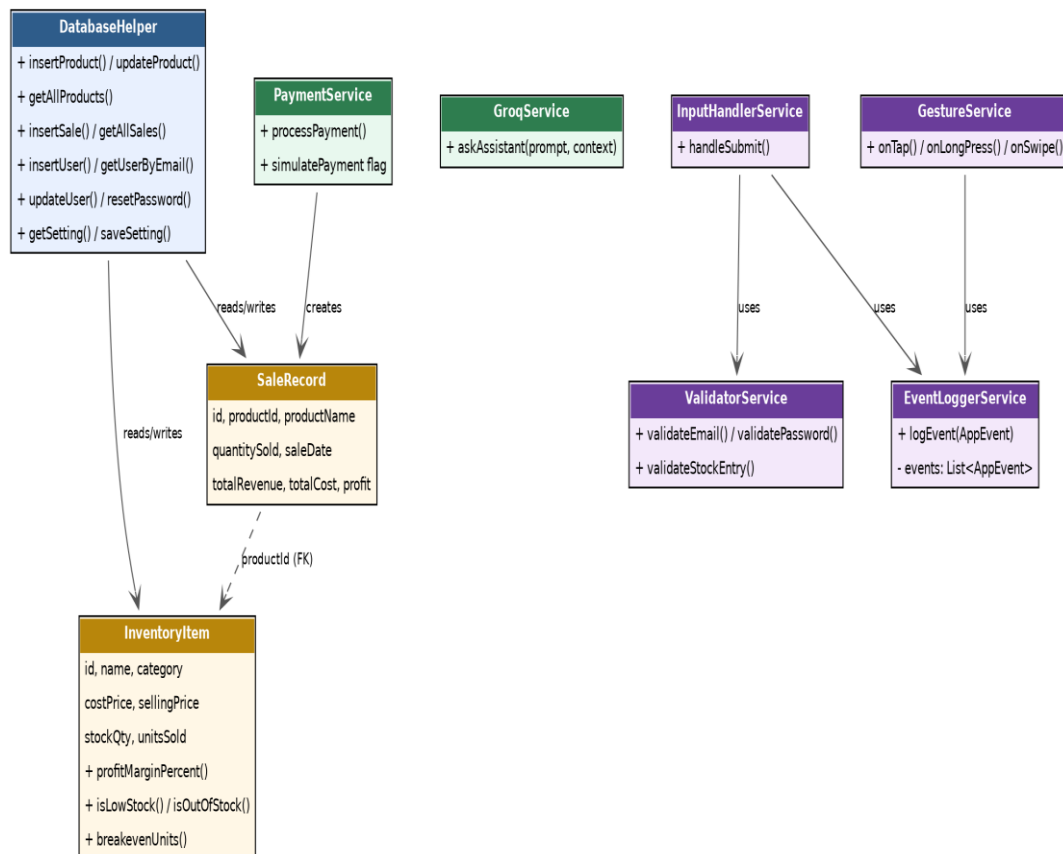


Figure 2 – Class Diagram: core models, persistence layer, and service classes.

6.3 Entity-Relationship Diagram

The database is normalised to Third Normal Form (3NF) across four tables. products and sales are linked by productId; settings holds app-level configuration; users is independent, storing hashed credentials and role information.

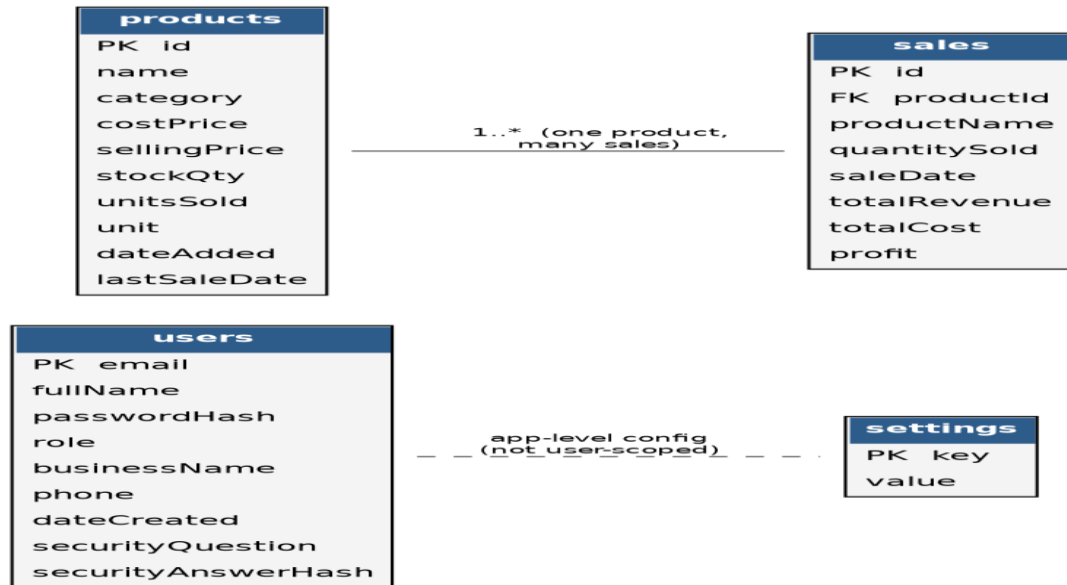


Figure 3 – Entity-Relationship Diagram of the local SQLite schema.

7. Implementation

The application was built in Flutter/Dart, with platform-conditional persistence: sqflite on Android/iOS, and sembast/sebast_web when compiled for the browser, behind a single DatabaseHelper interface so the rest of the app never needs to know which backend is active.

7.1 Authentication and Roles

Registration and login use hashed (SHA-256) credentials against the users table. Two recovery paths were implemented: a self-service flow where a user answers a stored security question, and an admin-assisted flow where a Merchant can reset an Operator's password directly.

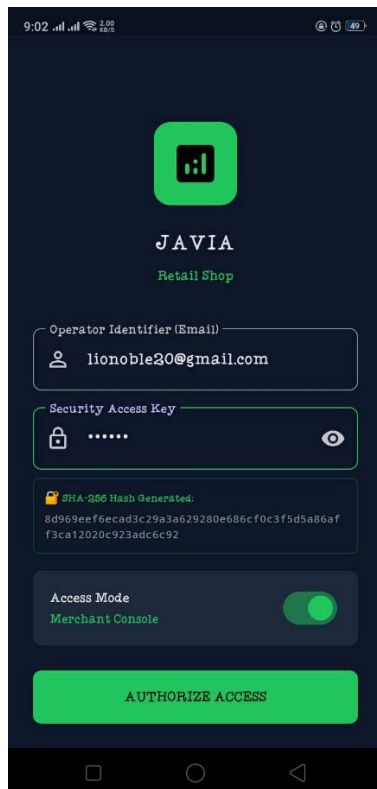


Figure 4 – Login screen, showing live SHA-256 hash generation used for credential verification.

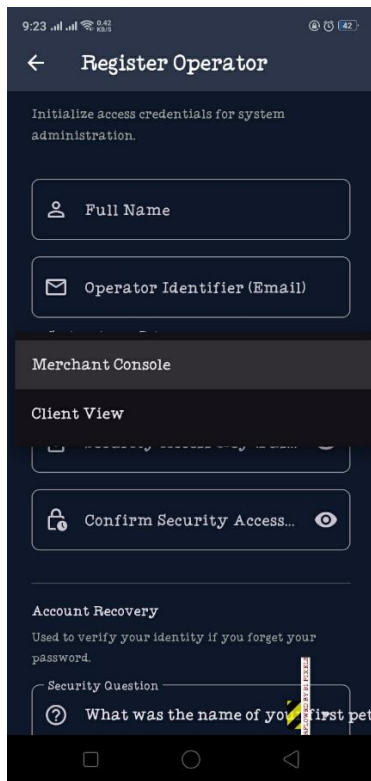


Figure 5 – Registration screen with role selection (Merchant Console / Client View).

7.2 Inventory and Sales

The Catalog screen lists all products with live stock and pricing, supports search and category filtering, and flags out-of-stock items. Sales are recorded against a product and immediately reflected in stock levels and the dashboard.

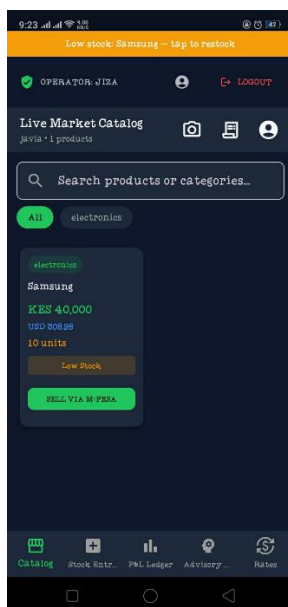


Figure 6 – Live Market Catalog, showing stock levels and category filtering.

7.3 Profit and Loss Dashboard

The dashboard aggregates sales data into a real-time profit and loss view, with daily/weekly/monthly/all-time filters, realized margin, and per-product breakdowns.

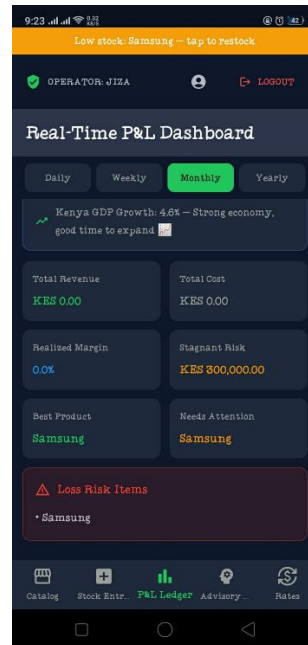


Figure 7 – Real-Time P&L Dashboard.

7.4 Payments

Sales can be recorded through M-Pesa via the IntaSend API, with a simulate/live toggle used during development and testing to avoid unnecessary real transactions while still exercising the full payment flow

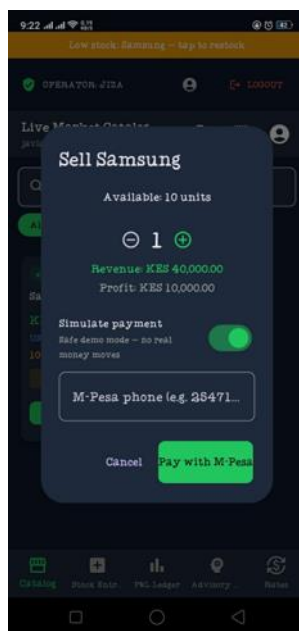


Figure 8 – Sale and payment screen with M-Pesa integration (Simulate/Live toggle).

7.5 AI Business Assistant

A chat interface (built on Groq's llama-3.3-70b-versatile model, chosen after early Gemini quota/billing issues) pulls the user's live inventory context into each prompt, letting it answer specific questions like current margins or which products are low on stock.

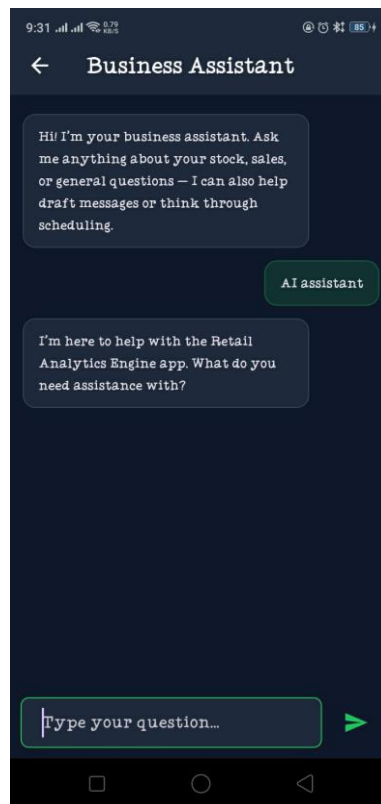


Figure 9 – AI Business Assistant chat interface.

7.6 Currency and Market Context

A dedicated screen shows live exchange rates (via the Open Exchange Rates API) and macroeconomic indicators (via the World Bank API) that feed into the Advisory Engine's recommendations, such as comparing realized margin against current inflation.

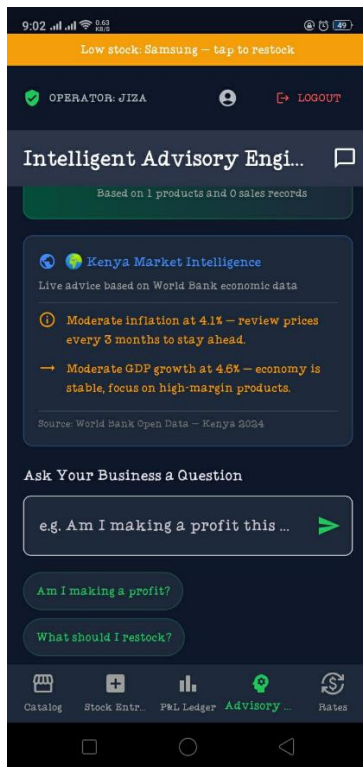


Figure 10 – Intelligent Advisory Engine, combining sales data with live economic indicators.

7.7 Device Features

Two native device features are integrated into real workflows rather than standalone demos: the camera (via `image_picker`) for capturing a product photo when adding stock, and GPS (via `geolocator/geocoding`) for tagging a stock entry's location — combined into a single 'Product Field Record' capture.

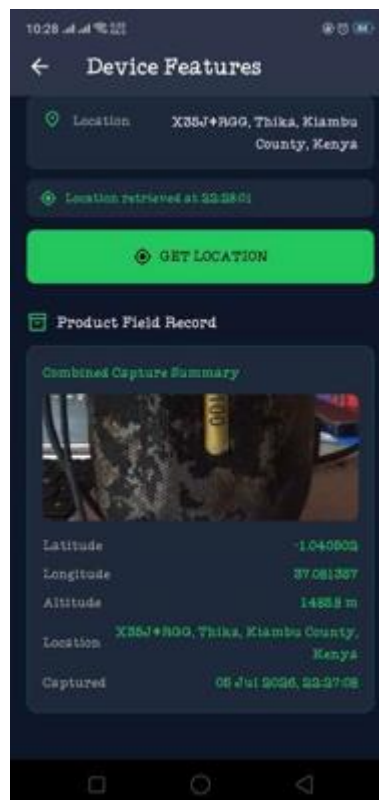


Figure 11 – Device Features screen: GPS location capture feeding into a Product Field Record.

8. Testing

Testing followed a structured approach: three JUnit-style unit tests were written against `InventoryItem` covering `profitMarginPercent`, `isLowStock/isOutOfStock`, and `breakevenUnits` — all passing. A full on-device debugging session was carried out on a physical CPH1903 (Android 8.1) test device using VS Code's Dart debugger, including a breakpoint set inside the AI Assistant's message-sending logic to step through and verify a live Groq API response. A real bug was found and fixed during this process: the AI Assistant screen referenced a non-existent quantity getter on `InventoryItem` where the actual field was `stockQty`.

- Unit tests: 3/3 passed (profit margin, stock status, breakeven calculation).
- Debugging: live breakpoint session confirmed correct request/response handling for the AI Assistant.
- Bug fixed: incorrect field reference (quantity → `stockQty`), caught by the compiler and verified live.

9. Performance Evaluation

Performance profiling was carried out with Flutter DevTools running against the app in `--profile` mode (not debug mode, which carries overhead that would distort the results) on the my oppo device.

- Frame rate: 56–60 FPS average during normal navigation, with no jank or shader-compilation stalls observed.
- Memory: stable at 13.3 MB total across two separate captures taken during active navigation, indicating no memory leak.
- CPU: a 432.3 ms profiling window (1,089 samples) showed time concentrated in expected native rendering calls rather than inefficient application logic.

Beyond profiling, a concrete optimization was identified and applied in `DatabaseHelper.updateUser()`: the mobile (sqlite) path previously read a user's full row, merged new values in Dart, and wrote the whole row back — two database round trips for what is typically a one or two-field change. This was replaced with a single targeted SQL UPDATE containing only the fields actually supplied, removing the extra read entirely while preserving the existing behaviour of leaving unspecified fields unchanged. The fix was verified on-device by updating a profile's name, business name, and phone number and confirming all three values persisted correctly.

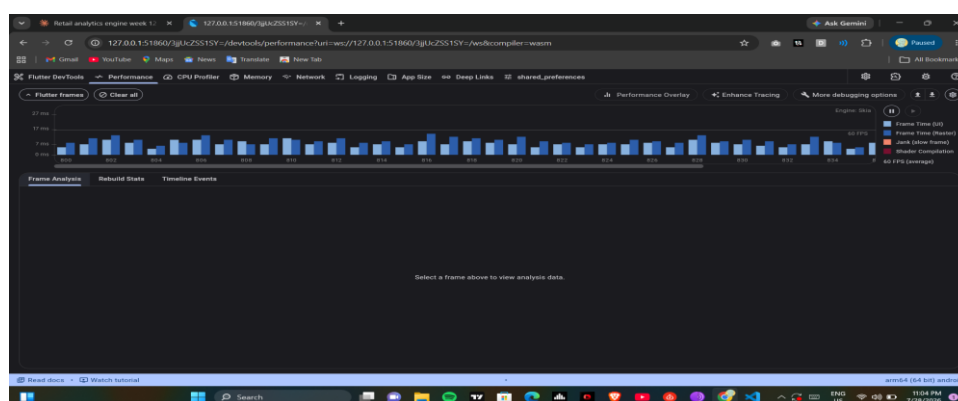


Figure 12 – DevTools Performance tab: 60 FPS average, no jank.

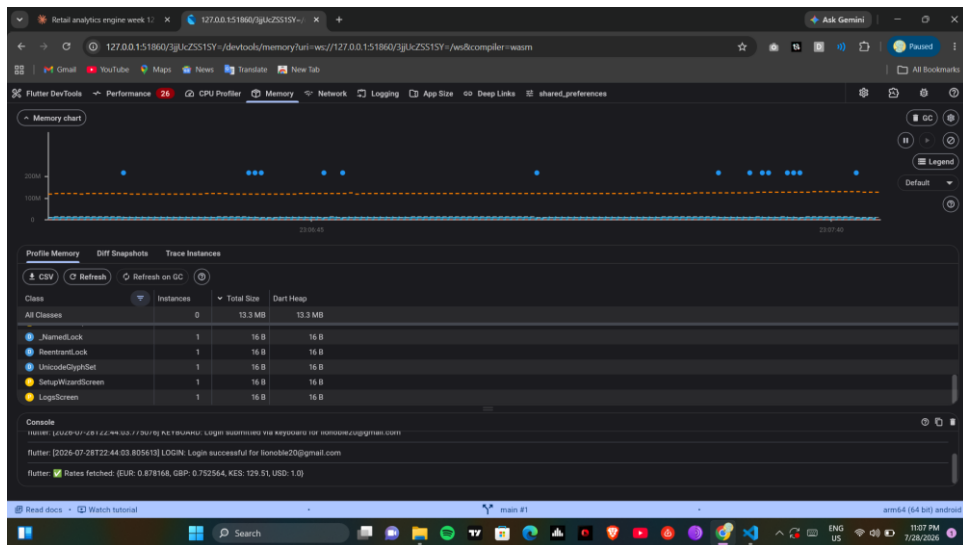


Figure 13 – DevTools Memory tab: stable 13.3 MB across navigation.

The screenshot shows the 'Operator Profile' screen. It contains the following fields and sections:

- Email:** lionoble20@gmail.com
- Merchant:** Merchant
- Full Name:** jiza
- Business Name:** oraimo
- Phone:** 0703358067
- Appearance:** Switch between dark and light interface themes. Dark Mode is enabled.
- Administration:** Merchant-only tools for managing operator accounts.

Figure 14 – Operator Profile after the updateUser() fix, confirming all fields persisted correctly.

10. Challenges Encountered

- Early AI integration used Gemini, which hit unresolved billing/quota limits; the assistant feature was migrated to Groq (free tier, OpenAI-compatible API) instead.
- A malformed leftover `--dart-define` argument in `.vscode/launch.json` silently blocked Debug-panel launches; traced and fixed during the Week 12 debugging session.

- Payment testing on the web build hit CORS restrictions against the IntaSend API, which does not affect the Android build but required a simulate-mode fallback for web testing.
- A duplicated main.dart definition caused a build conflict earlier in the project and had to be traced and removed.
- Keystore/signing configuration required a repeat attempt after an initial certificate mismatch during the Week 11 deployment preparation.

11. Recommendations

- Add push notifications for low-stock alerts, so operators do not need to open the app to notice a stockout risk.
- Extend the Merchant/Operator model to support multiple branches or outlets under one business account.
- Add barcode/QR scanning as an alternative to manual product entry, to speed up stock intake for larger catalogs.
- Add an offline sync queue for payments made while the device is offline, to be reconciled with IntaSend once connectivity returns.

12. Conclusion

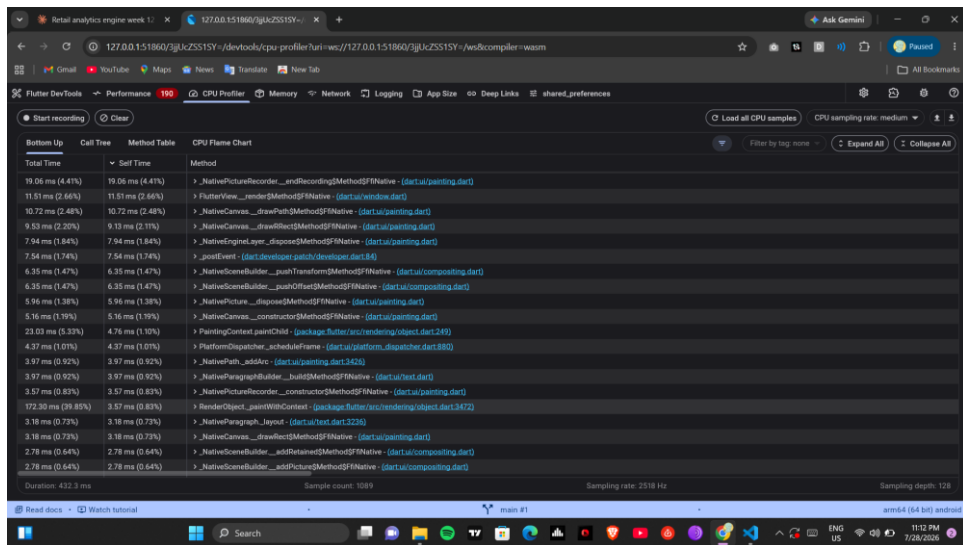
The Retail Analytics Engine met its stated objectives: it provides secure authentication, a complete inventory and sales workflow, real payment integration, live currency and economic context, an AI-assisted advisory feature, and native device integrations, all validated through structured testing and performance profiling, and packaged as a signed, installable Android release. Because it was built iteratively across the semester rather than as a single final push, each stage — data modelling, OOP service design, API integration, testing, and optimization — was validated on real, working functionality rather than a static plan. The result is a general-purpose retail tool that does not assume any one category of business, and that a small operator could realistically install and start using on their own phone.

13. References

Flutter documentation — <https://docs.flutter.dev>
 Dart language documentation — <https://dart.dev/guides>
 sqflite package — <https://pub.dev/packages/sqflite>
 sembast / sembast_web packages — <https://pub.dev/packages/sembast>
 Groq API documentation — <https://console.groq.com/docs>
 IntaSend API documentation — <https://developers.intasend.com>
 Open Exchange Rates API — <https://openexchangerates.org/documentation>
 World Bank Open Data API — <https://data.worldbank.org>
 Flutter DevTools documentation — <https://docs.flutter.dev/tools/devtools>
 image_picker and geolocator/geocoding packages — <https://pub.dev>

14. Appendices

Additional evidence screenshots not already shown inline above.



Appendix A – CPU Profiler, Bottom-Up view (432.3 ms sampled).